

School ERP Software Architecture Document (SAD)

Project Status: In Development (Version 1.0)

1. Vision

Build a modern, secure, configurable, and scalable School ERP platform that enables schools to manage their entire operation through one unified system.

The platform is designed for long-term evolution, beginning with a single-school deployment and later expanding to support multiple schools and campuses.

2. Core Principles

- Schools come first.
- Solve real problems.
- Security by default.
- Simplicity over complexity.
- Configuration over hardcoding.
- Performance matters.
- Every feature must have a clear purpose.
- Code should be understandable years later.
- The system should be extensible without major rewrites.

3. Technology Stack

Frontend

- React.js
- React Router
- Plain CSS
- Axios
- Socket.IO Client

- Chart.js

Backend

- Node.js
- Express.js
- Socket.IO
- MySQL
- JWT
- bcrypt
- Multer
- Nodemailer
- PDFKit
- ExceUS
- docx
- QRCode
- Express Validator
- Helmet
- CORS
- Morgan
- dotenv

Database

- MySQL
- Raw SQL only (No ORM)

4. Folder Structure

/project

/client

/server

/database

/docs

/assets

5. Architecture

Client

↓

REST API

↓

Express Server

↓

Business Logic

↓

MySQL Database

Socket.IO provides all real-time communication.

6. Security Standards

- Passwords are hashed using bcrypt.
- JWT authentication.
- Role-Based Access Control.
- Input validation on every endpoint.
- SQL injection prevention.

- Environment variables for secrets.
- Audit logs for sensitive actions.
- HTTPS in production.
- Security headers using Helmet.

7. Coding Standards

- ES Modules only.
- Descriptive variable names.
- Small reusable functions.
- Consistent naming.
- Avoid duplicate code.
- Prefer composition over repetition.
- Business logic belongs in controllers/services.
- Never expose secrets.

8. Database Standards

Every major table should include:

- id
- created_at
- updated_at
- created_by (where appropriate)
- updated_by (where appropriate)
- status (where appropriate)

Soft deletion should be used for important records.

9. API Standards

- RESTful endpoints.

- Consistent JSON responses.
- Appropriate HTTP status codes.
- Pagination for large datasets.
- Filtering and searching where applicable.

10. Authentication

- Login
- Logout
- Password Reset
- Refresh Tokens
- Session Tracking
- Permission Checks

11. Logging

Log:

- Login
- Logout
- Failed login attempts
- Record creation
- Record updates
- Record deletion
- Permission changes
- Financial transactions

12. Real-Time Features

Socket.IO will power:

- Messaging

- Notifications
- Live attendance
- Dashboard updates
- Announcement delivery
- Online status
- Typing indicators

13. File Management

Support:

- Images
- PDFs
- Word Documents
- Excel Files
- Student Photos
- Teacher Photos

14. Reporting

Export formats:

- PDF
- Word (.docx)
- Excel (.xlsx)
- CSV

Reports should be printable and customizable.

15. UI Principles

- Responsive by default.

- Accessible.
- Fast.
- Consistent.
- Professional.
- Minimal learning curve.

16. Customization

Schools should be able to configure:

- Academic structure
- Grading systems
- Terms/Semesters
- Branding
- Roles
- Permissions
- Notification rules
- Report layouts
- Certificates
- Student custom fields

Avoid hardcoding wherever practical.

17. Future Roadmap

- Multi-campus
- Multi-school
- Mobile applications
- AI-powered insights
- Biometric attendance
- Face recognition
- Payment gateway integration
- SMS and WhatsApp integration
- Plugin marketplace

- Offline support

18. Mission Statement

We are not building software with many features.

We are building the digital operating system for schools.

Every design decision should make schools more efficient, more organized, and better equipped to educate their students.

Guiding Principle

«Build for today's schools while designing for tomorrow's possibilities.»